Towards Differentiable Creativity

Dmitri Tkatch

Abstract

Deep learning-based language models have revolutionized the field of artificial intelligence. Yet perhaps their biggest shortcoming is their partial inability to think creatively. We hypothesize that creativity is partially a byproduct of the more general process of synaptic feedback-based thinking. We attribute modern language model creativity deficits to this lack of synaptic feedback at inference time. We find that a small language model (Qwen2.5-0.5B-Instruct) tested on math word problems (GSM8k) performs better when trained on its own sharpened output distribution (temperature 1.0 outputs with temperature 0.6 targets) on a per-generated token basis. Code is available at `https://github.com/eternalyze1/differentiable_creativity` as well as Appendix A.

1 Motivation

In a recent YouTube video Rich Sutton describes the creative deficits of language models as well as the ingredients of creativity. Briefly, they are: variation, evaluation, and selective retention. While we generally agree with these essentials, one could argue that sampling produces variation, that RLVR/self-correction is a form of evaluation, and that training on rewards/attentive in-context learning is a form of self-retention; and thereby conclude that language models are creative at train/test time. Of course, as is almost always the case in machine learning, it is not enough to simply have the ingredients, they also have to be put into practice in an efficient and effective way.

When one is writing a proof there is a feeling of both reward and tunnel vision (or focus) even before the correctness and validity of the proof is assured. Moreover, there is a feeling of learning that stays with you even if the proof is never itself completed. We wish to capture (or formalize) this feeling. That is why we seek a minimalist form of per-token test-time self-distillation. This feeling arises immediately when one attempts a problem, so it must be on a per-token basis. And it arises without any form of verifiable reward, so it must be a form of self-distillation (or based on some intrinsic motive). This feeling also dissipates almost immediately after one stops working on the problem. Thus it is reasonable to assume that it happens throughout test-time.

In particular, we hypothesize that the signal must be more dense than that of typical RLVR. And so we take the position that one should delegate as much of RL as possible to (self-) SL and hypothesize that evolution takes a similar position.

Preprint.

2 Results

Table 1: GSM8K results across three runs. Sampling uses $T = 0.6$, top-$p = 0.95$. The baseline is Qwen2.5-0.5B-Instruct and DCLM (for Differentiable Creativity Language Model) is Qwen2.5-0.5B-Instruct with sharpened test-time self-distillation. For DCLM the model parameters are restored after every problem. SGD optimizer is used for self-distillation.

Run	Method	LR	Acc	Avg Tokens	Time (s)
Run 1	Baseline	–	44/100	301	3072
(greedy)	DCLM	10^{-5}	45/100	306	9152
Run 2	Baseline	–	39/100	323	2737
(sampling)	DCLM	10^{-5}	43/100	335	9040
Run 3	Baseline	–	44/100	305	823
(sampling)	DCLM	10^{-6}	46/100	306	7810

References

[1] Bishop, C.M. & Bishop, H. (2024) *Deep Learning: Foundations and Concepts.* Cham: Springer.

[2] Schmidhuber, J. (2010) Formal theory of creativity, fun, and intrinsic motivation (1990–2010). *IEEE Transactions on Autonomous Mental Development* **2**(3):230–247.

[3] Sutton, R. (2026) Rich Sutton on "AI creativity & discovery." YouTube video. `https://www.youtube.com/watch?v=K5LAFEjTlBA`.

[4] Mohamed, S.& Rezende, D.J. (2015) Variational information maximisation for intrinsically motivated reinforcement learning. Proceedings of the 29th Conference on Neural Information Processing Systems (NIPS 2015). arXiv:1509.08731.

[5] Hübotter, J., Lübeck, F., Behric, L., Baumann, A., Bagatella, M., Marta, D., Hakimi, I., Shenfeld, I., Kleine Buening, T., Guestrin, C. & Krause, A. (2026) Reinforcement learning via self-distillation. arXiv preprint arXiv:2601.20802.

A Implementation

The complete DCLM evaluation script is shown below.

```
import sys, io
sys.stdout = io.TextIOWrapper(sys.stdout.buffer, encoding='utf-8')

import torch
import torch.nn.functional as F
from transformers import AutoTokenizer, AutoModelForCausalLM
from datasets import load_dataset
import time, argparse, re

parser = argparse.ArgumentParser()
parser.add_argument("--model", type=str, default="Qwen/Qwen2.5-0.5B-Instruct")
parser.add_argument("--lr", type=float, default=1e-5, help="DCLM learning rate")
parser.add_argument("--distill_temp", type=float, default=0.6, help="Teacher temperature for distillation")
parser.add_argument("--dataset", type=str, default="gsm8k", choices=["gsm8k", "math"])
parser.add_argument("--max_new_tokens", type=int, default=1024)
parser.add_argument("--num", type=int, default=50)
args = parser.parse_args()

dtype = torch.float16

print(f"Loading {args.model}...")
```

```
t0 = time.time()
tokenizer = AutoTokenizer.from_pretrained(args.model, trust_remote_code=True)
model = AutoModelForCausalLM.from_pretrained(
    args.model, trust_remote_code=True,
    torch_dtype=dtype, device_map="auto",
)
if tokenizer.pad_token is None:
    tokenizer.pad_token = tokenizer.eos_token
model_device = next(model.parameters()).device
print(f"Loaded␣in␣{time.time()-t0:.1f}s␣on␣{model_device}")

# Save original weights to restore between DCLM problems
original_state = {k: v.clone() for k, v in model.state_dict().items()}

V = model.config.vocab_size

print(f"Loading␣{args.dataset.upper()}...")
if args.dataset == "gsm8k":
    ds = load_dataset("gsm8k", "main", split=f"test[:{args.num}]")
    q_key, a_key = "question", "answer"
else:
    ds = load_dataset("nlile/hendrycks-MATH-benchmark", split=f"test[:{args.
        num}]")
    q_key, a_key = "problem", "answer"

def sample(logits, temperature=0.6, top_p=0.95):
    logits = logits / temperature
    if top_p < 1.0:
        sorted_logits, sorted_indices = torch.sort(logits, descending=True,
            stable=True)
        cum_probs = torch.cumsum(F.softmax(sorted_logits, dim=-1), dim=-1)
        sorted_indices_to_remove = cum_probs > top_p
        sorted_indices_to_remove[..., 1:] = sorted_indices_to_remove[...,
            :-1].clone()
        sorted_indices_to_remove[..., 0] = False
        indices_to_remove = sorted_indices_to_remove.scatter(1,
            sorted_indices, sorted_indices_to_remove)
        logits[indices_to_remove] = float('-inf')
    return torch.multinomial(F.softmax(logits, dim=-1), num_samples=1)

def extract_answer(text):
    m = re.search(r"\\boxed\{([^}]+)\}", text)
    if m:
        return m.group(1).strip()
    m = re.search(r"####\s*(-?\d+(?:\.\d+)?)", text)
    if m:
        return m.group(1)
    return None

def extract_expected(text):
    """Extract␣numeric␣answer␣from␣GSM8K␣answer␣field␣(format:␣'explanation␣
        ####␣NUMBER')."""
    m = re.search(r"####\s*(-?\d+(?:\.\d+)?)", text)
    if m:
        return m.group(1)
    return text.strip()

def generate_baseline(prompt_ids, max_new_tokens=512):
    model.eval()
    past = None
    generated = prompt_ids.clone()
    hit_limit = True
    for _ in range(max_new_tokens):
        with torch.no_grad():
            outputs = model(
                generated if past is None else generated[:, -1:],
                past_key_values=past, use_cache=True,
            )
```

```
            past = outputs.past_key_values
            logits = outputs.logits[:, -1, :]
        token = sample(logits[:, :V])
        if token.item() == tokenizer.eos_token_id:
            hit_limit = False
            break
        generated = torch.cat([generated, token], dim=-1)
    return generated[0], hit_limit

def generate_dclm(prompt_ids, max_new_tokens=512, lr=1e-5, distill_temp=2.0):
    """DCLM: training at inference time via knowledge distillation.
    Forward with grad tracking, compute a 'teacher' distribution from
    logits at higher temperature, then backprop CE(teacher || softmax(logits)
    )
    through the full network and take an SGD step. This trains the model
    to reproduce a more uniform (creative) distribution at each step.
    Uses KV cache with truncated BPTT (past tensors detached)."""
    model.eval()
    with torch.no_grad():
        model.load_state_dict(original_state)
    generated = prompt_ids.clone()
    hit_limit = True
    optimizer = torch.optim.SGD(model.parameters(), lr=lr)

    # Build prompt KV cache (no_grad)
    with torch.no_grad():
        past = model(generated, use_cache=True).past_key_values

    for _ in range(max_new_tokens):
        outputs = model(generated[:, -1:], past_key_values=past, use_cache=
            True)
        logits = outputs.logits[:, -1, :V]

        # Sample token (detached, just for continuing generation)
        token = sample(logits.detach())

        if token.item() == tokenizer.eos_token_id:
            hit_limit = False
            generated = torch.cat([generated, token], dim=-1)
            break

        # Sharpened self-distillation: CE(softmax(logits/T) || softmax(logits
            ))
        # T<1 = sharper teacher, pulling model toward more confident
            predictions
        with torch.no_grad():
            target = F.softmax(logits / distill_temp, dim=-1)
        loss = F.cross_entropy(logits, target)
        loss.backward()
        optimizer.step()
        optimizer.zero_grad()

        # Detach KV cache after backward to prevent graph chaining
        for i in range(len(past.layers)):
            past.layers[i].keys = past.layers[i].keys.detach()
            past.layers[i].values = past.layers[i].values.detach()

        generated = torch.cat([generated, token], dim=-1)

    return generated[0], hit_limit

results = {"baseline": {}, "dclm": {}}
for label, gen_fn, store in [
    ("BASELINE", generate_baseline, results["baseline"]),
    ("DCLM", generate_dclm, results["dclm"]),
]:
    correct = 0
    total = 0
```

```
    truncated = 0
    total_gen_tokens = 0
    t_start = time.time()
    for i, example in enumerate(ds):
        question = example[q_key]
        answer = example[a_key]
        messages = [{"role": "user", "content": f"{question}\nPlease␣reason␣
            step␣by␣step,␣and␣put␣your␣final␣answer␣within␣\\boxed{{}}."}]
        inputs = tokenizer.apply_chat_template(
            messages, add_generation_prompt=True, tokenize=True,
            return_dict=True, return_tensors="pt",
        ).to(model_device)
        prompt_len = inputs["input_ids"].shape[1]
        if label == "DCLM":
            full_ids, hit_limit = gen_fn(inputs["input_ids"], max_new_tokens=
                args.max_new_tokens, lr=args.lr, distill_temp=args.
                distill_temp)
        else:
            full_ids, hit_limit = gen_fn(inputs["input_ids"], max_new_tokens=
                args.max_new_tokens)
        gen_len = full_ids.shape[0] - prompt_len
        total_gen_tokens += gen_len
        output_text = tokenizer.decode(full_ids[prompt_len:],
            skip_special_tokens=True)
        predicted = extract_answer(output_text)
        expected = extract_expected(str(answer))
        if hit_limit:
            truncated += 1
            print(f"␣␣[WARNING]␣Hit␣max_new_tokens␣limit␣({args.
                max_new_tokens})␣-␣generation␣truncated!")
        correct += 1 if (predicted is not None and expected is not None and
            predicted == expected) else 0
        total += 1
        q_safe = question.replace("\u2019", "'").replace("\u2018", "'").
            replace("\u201c", '"').replace("\u201d", '"').replace("\u2014", "
            --").replace("\u2013", "-")
        a_safe = output_text.replace("\u2019", "'").replace("\u2018", "'").
            replace("\u201c", '"').replace("\u201d", '"').replace("\u2014", "
            --").replace("\u2013", "-")
        print(f"[{label}]␣#{i}␣({gen_len}␣tok)␣pred={predicted}␣expected={
            expected}␣{'[OK]'␣if␣predicted␣==␣expected␣else␣'[FAIL]'}")
        print(f"␣␣Q:␣{q_safe}")
        print(f"␣␣A:␣{a_safe}")
        print()
    acc = correct / total * 100 if total > 0 else 0
    elapsed = time.time() - t_start
    avg_len = total_gen_tokens / total if total > 0 else 0
    store["accuracy"] = acc
    store["correct"] = correct
    store["total"] = total
    store["truncated"] = truncated
    store["avg_len"] = avg_len
    store["time"] = elapsed
    print(f"[{label}]␣Done:␣{correct}/{total}={acc:.1f}%␣({elapsed:.0f}s)")

print("\n" + "=" * 50)
print("RESULTS")
print("=" * 50)
for label, store in [("BASELINE", results["baseline"]), ("DCLM", results["
    dclm"])]:
    if store:
        print(f"{label}:␣{store['correct']}/{store['total']}={store['accuracy
            ']:.1f}%␣|␣truncated={store['truncated']}/{store['total']}␣|␣
            avg_len={store['avg_len']:.0f}␣tokens␣({store['time']:.0f}s)")
```